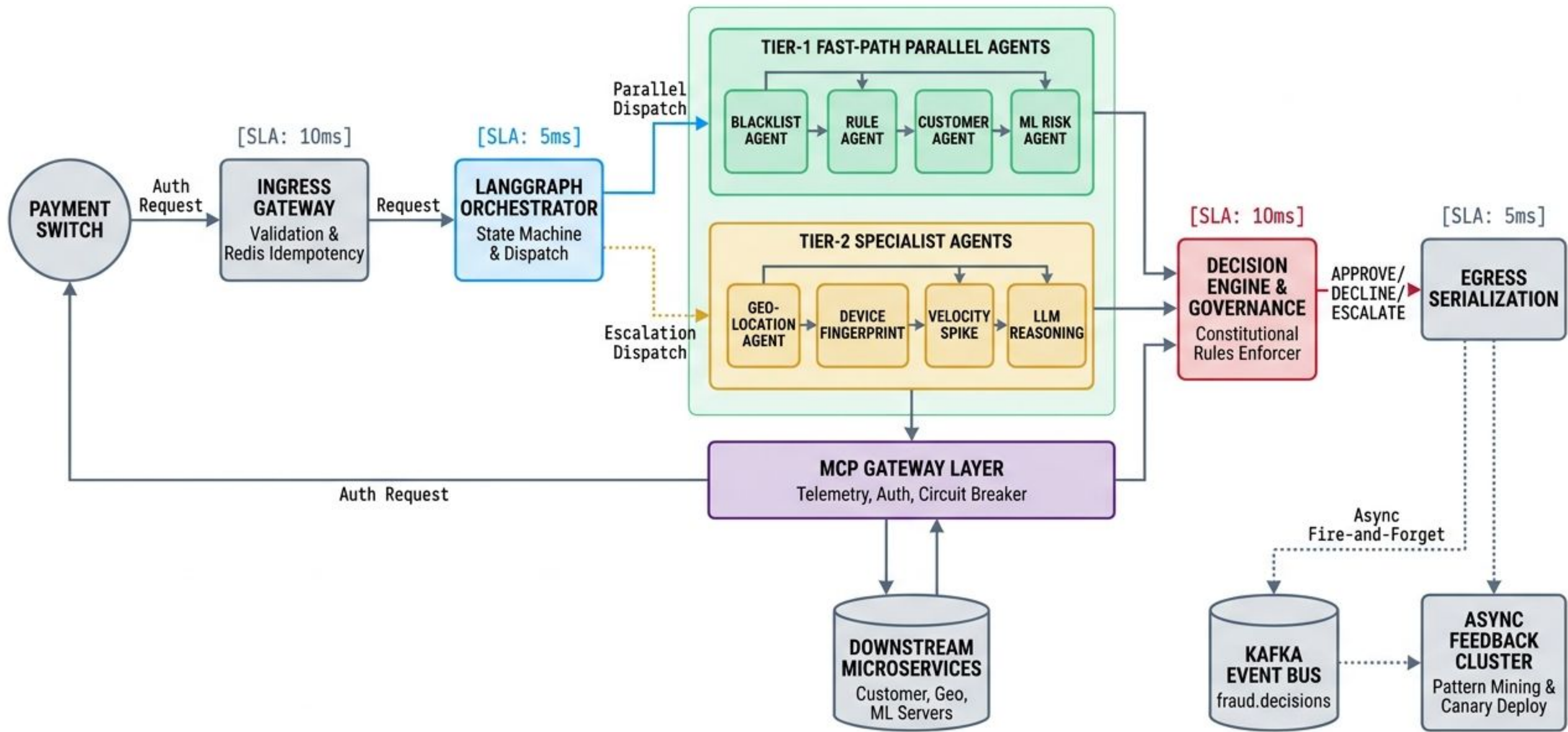


SYSTEM TOPOLOGY & ARCHITECTURE FLOW



Latency Budget & SLA Allocation Timeline

Execution Phase	Latency Budget	Execution Model	Cumulative SLA Visualization (100ms Total)
Ingress Parsing	10ms	Sequential	████████.....
Orchestrator Setup	5ms	Sequential	████████.....
Rule Agent	5ms	Parallel	████████.....
Blacklist Agent	10ms	Parallel	████████.....
Customer Agent	20ms	Parallel	████████.....
ML Risk Agent	25ms	Parallel	████████.....
Decision Engine	10ms	Sequential	████████.....
Egress Comm.	5ms	Sequential	████████.....
Reserve Buffer	10ms	Safety	████████<-- GC Jitter

Total Active Pipeline (Fast-Path): $10 + 5 + \text{Max}(5, 10, 20, 25) + 10 + 5 = 55\text{ms}$
Tier-2 / Specialist Allocation: Operates dynamically within the remaining 35ms (pre-buffer) to ensure the 100ms ceiling is never breached. Exceeding 80ms total triggers the Guardrail Agent fallback.

Model Context Protocol (MCP) Gateway Layer

Core Concept: Agents do not possess direct network access. All agent capabilities are routed through standardized JSON-RPC interfaces via the MCP Gateway Layer.

1. Agent Origin 	2. Gateway Middleware (MCP) 	3. Downstream Execution 
Tool Abstraction Agents query standardized schemas without knowing endpoint URIs or transport protocols. Budget Propagation Agents attach their remaining SLA timestamp to the payload header. Request Context 'NormalizedTransaction' injected dynamically.	Access Control Filters Strict RBAC validation. Ensures the LLM Agent cannot execute high-risk mutation tools. Circuit Breakers Gracefully degrades and returns heuristic defaults if external services timeout or rate-limit. Distributed Tracing Injects correlation IDs. Comprehensive logs tracing every query hop for the Guardrail Agent.	Service Mesh Routes to discrete backend servers ('ml_server', 'geo_server', 'customer_server'). Database Queries Executes complex joins on underlying data stores. Response Serialization Standardized JSON responses returned back through the gateway.

Agent Profile | Blacklist Agent

Implementation: `blacklist_agent.py` | `BlacklistAgent`

 Operational Mechanisms	 MCP Tools Invoked (<code>blacklist_server</code>)	 I/O Contract	
Identifies known bad actors in real-time by cross-referencing transaction attributes against high-speed cache blacklists. Immediately flags compromised instruments or entities.	<code>check_card_blacklist(card_id)</code>: Verifies if instrument is stolen/compromised. <code>check_device_blacklist(device_id)</code>: Checks device signature for prior fraud. <code>check_merchant_blacklist(merchant_id)</code>: Matches merchant identity to fraudulent endpoints.	Input Context (Normalized Transaction)	Output Schema
		<code>card_id</code> (str) <code>device_id</code> (str) <code>merchant_id</code> (str)	<code>blacklisted</code> (bool) - True if any entity hits <code>card_blacklisted</code> (bool) <code>device_blacklisted</code> (bool) <code>merchant_blacklisted</code> (bool) <code>source</code> (str: 'card' 'device' 'merchant' '') <code>match_type</code> (str: 'exact' 'none') <code>evidence</code> (List[AgentEvidence])

Agent Profile | Rule Agent

[Execution Tier: TIER-1 FAST-PATH]

[Latency Budget: 5ms]

Implementation: rule_agent.py | RuleAgent

 Operational Mechanisms	 Evaluated Rules (In-Memory)	 I/O Contract				
Executes deterministic, compliance-driven business rules. Operates with extreme reliability (<2ms) as it requires zero external database or network lookups.	1. Sanctioned Country: Critical violation (unconditional decline) for KP, IR, SY, CU, SD. 2. High-Risk Check: Flags risky locations and merchant categories (casinos, crypto). 3. Channel Limits: Enforces maximums (Online: \$10k, POS: \$5k, ATM: \$2k, Mobile: \$10k), \$2k, ATM: \$1C. 4. Suspicious Hours: Flags activity between 1:00 AM - 5:00 AM UTC. 5. High Value: Flags any transaction >\$25,000.	<table><tr><th>Input Context (NormalizedTransaction)</th><th>Output Schema</th></tr><tr><td> - country (str) - amount_usd (float) - channel (str) - merchant_category (str) - timestamp (datetime) </td><td> - violation (bool) - True if any rule broken - rule_name (str) - Worst rule violated - rule_severity (str: 'critical' 'high' ...) - violations (List[str]) - evidence (Metadata for audit trail) </td></tr></table>	Input Context (NormalizedTransaction)	Output Schema	- country (str) - amount_usd (float) - channel (str) - merchant_category (str) - timestamp (datetime)	- violation (bool) - True if any rule broken - rule_name (str) - Worst rule violated - rule_severity (str: 'critical' 'high' ...) - violations (List[str]) - evidence (Metadata for audit trail)
Input Context (NormalizedTransaction)	Output Schema					
- country (str) - amount_usd (float) - channel (str) - merchant_category (str) - timestamp (datetime)	- violation (bool) - True if any rule broken - rule_name (str) - Worst rule violated - rule_severity (str: 'critical' 'high' ...) - violations (List[str]) - evidence (Metadata for audit trail)					

Agent Profile | Customer Profile Agent

Implementation: customer_agent.py | CustomerAgent

[Execution Tier: TIER-1 FAST-PATH]

[Latency Budget: 20ms]

⚙️ Operational Mechanisms

Analyzes customer profile data, spending patterns, and account maturity.

Calculates spending anomaly scores (z-scores) relative to the customer's historical average:

$$z = \frac{\text{Amount} - \text{Average Amount}}{\text{Standard Deviation}}$$

(z >= 2.0 = Elevated Risk; z >= 3.0 = Immediate Escalation)

🔧 MCP Tools Invoked (customer_server)

1. `get_customer_profile(customer_id)`: Fetches trust tier, account age, and past fraud flags.
2. `get_spending_stats(customer_id)`: Fetches mean transaction amount and standard deviation.

↔️ I/O Contract

Input Context (NormalizedTransaction)	Output Schema
• <code>customer_id</code> (str) • <code>amount_usd</code> (float)	• <code>trust_tier</code> (str: 'platinum' 'standard' 'new') • <code>avg_transaction_amount</code> (float) • <code>spending_anomaly</code> (float z-score) • <code>account_age_days</code> (int) • <code>total_transactions</code> (int) • <code>fraud_history</code> (int count) • <code>evidence</code> (Detailed metrics logs)

Agent Profile | ML Risk Agent

[Latency Budget: 25ms]

Implementation: ml_risk_agent.py | MLRiskAgent

⚙️ Operational Mechanisms

Fetches statistical predictions from a serialized `ii` prediction from a serialized Random Forest classifier trained on historical transaction datasets.

Features automatic deterministic heuristic fallback if the model service goes offline.

🔧 MCP Tools Invoked (ml_server)

- `predict_fraud_score(...)`: Supplies transaction features and returns a probability score. `and` returns a probability score `[0.0, 1.0]`.

Fallback Heuristic

Base 0.1, +0.2 (>\$5k), +0.25 (High-risk country), +0.15 (High-risk merchant), +0.1 (Night hours).

↔️ I/O Contract

Input Context (NormalizedTransaction)	Output Schema
- amount_usd, channel - merchant_category, country - timestamp - is_high_risk_country - is_high_risk_merchant	- risk_score (float <code>[0.0, 1.0]</code> None) - model_version (str: e.g., 'rf-v1.2' or 'heuristic') - feature_importances (Dict for SHAP explainability) - evidence (Inference summaries)

Agent Profile | Geo-Location Agent

Implementation: `geo_agent.py` | GeoAgent

[Execution Tier: TIER-2 SPECIALIST]
[Latency Budget: 15ms]

 Operational Mechanisms

Performs geographical verification with a strict focus on identifying "impossible travel" scenarios by analyzing elapsed time and elapsed time and **Haversine distance** between consecutive transactions.

Flags if **Required Speed > 900 km/h**.

 MCP Tools Invoked (`geo_server`)

- `get_last_location(customer_id)`: Retrieves coordinates and timestamp of previous transaction.
- `calculate_distance(lat1, lon1, lat2, lon2)`: Computes Haversine distance in kilometers.

 I/O Contract

Input Context (NormalizedTransaction)	Output Schema
<code>customer_id</code> (str) <code>country</code> (str)	<code>distance_km</code> (float) <code>time_since_last_hours</code> (float) <code>required_speed_kmh</code> (float) <code>impossible_travel</code> (bool) <code>last_country</code> (str) <code>cross_border</code> (bool) <code>evidence</code> (Coordinates, speed calcs)

Agent Profile | Device Fingerprint Agent

[Execution Tier: TIER-2 SPECIALIST]

[Latency Budget: 10ms]

Implementation: device_agent.py | DeviceAgent

 Operational Mechanisms	 MCP Tools Invoked (device_server)	 I/O Contract				
Evaluates the physical device signature. Identifies device age, trust tier, and flags anomalies like credential stuffing (device sharing). Base trusted risk is 0.05. Modifiers: New Device (+0.4), Age < 7 days (+0.2), Shared with N customers (+0.3 * min(N/3, 1.0)).	1. get_device_profile(device_id, customer_id): Determines account association maturity. 2. check_device_sharing(device_id): Checks if used by multiple distinct customer IDs in a short window.	<table><tr><th>Input Context (NormalizedTransaction)</th><th>Output Schema</th></tr><tr><td> - device_id (str) - customer_id (str) </td><td> - new_device (bool) - device_age_days (int) - shared_device (bool) - device_risk (float [0.0, 1.0]) - anomalies (List[str] e.g., 'shared_4_customers') - evidence (Trust level logs) </td></tr></table>	Input Context (NormalizedTransaction)	Output Schema	- device_id (str) - customer_id (str)	- new_device (bool) - device_age_days (int) - shared_device (bool) - device_risk (float [0.0, 1.0]) - anomalies (List[str] e.g., 'shared_4_customers') - evidence (Trust level logs)
Input Context (NormalizedTransaction)	Output Schema					
- device_id (str) - customer_id (str)	- new_device (bool) - device_age_days (int) - shared_device (bool) - device_risk (float [0.0, 1.0]) - anomalies (List[str] e.g., 'shared_4_customers') - evidence (Trust level logs)					

Agent Profile | Velocity Spike Agent

[Execution Tier: TIER-2 SPECIALIST]

[Latency Budget: 10ms]

Implementation: velocity_agent.py | VelocityAgent

⚙️ Operational Mechanisms

Tracks high-frequency bursts and spending velocity anomalies over rolling time windows.

Triggers on:

- **Hourly Burst:** ≥ 10 txns or $\geq \$5,000$ / 1 hr.
- **Daily Burst:** ≥ 30 txns / 24 hrs.
- **Acceleration:** Hourly spend $> 3.0\times$ baseline avg.
- **Merchant Diversity:** ≥ 5 distinct merchants / 1 hr.

🔧 MCP Tools Invoked (velocity_server)

1. **get_transaction_velocity(customer_id):** Count of transactions (1h/24h) and merchant diversity.
2. **get_amount_velocity(customer_id):** Sum of transacted amount (1h/24h) vs hourly baseline.

↔️ I/O Contract

Input Context (NormalizedTransaction)	Output Schema
• customer_id (str)	• transactions_last_hour (int) / last_day (int) • amount_last_hour (float) / last_day (float) • burst (bool) • amount_acceleration (float) • distinct_merchants_last_hour (int) • evidence (Detailed logs of burst triggers)

Agent Profile | LLM Reasoning Agent

[Execution Tier: TIER-2 SPECIALIST]

[Latency Budget: 20ms]

⚠️ [POLICY OVERRIDE RESTRICTION]: Advisory Only. Unauthorized to make final block/approve network decisions.

Implementation: llm_reasoning.py | LLMReasoningAgent

 **Operational Mechanisms**

A lightweight fallback helper utilizing the Google Antigravity (AGY) SDK to perform **cognitive synthesis** of evidence gathered from preceding agents. Handles complex edge-cases that evade rigid rules.

 **Mechanisms & Integration**

- Constructs a structured prompt detailing transaction attributes and Tier 1 + Specialist outputs.
- Invokes **google.antigravity.Agent** endpoint.
- Parses structured JSON matching a strict **RiskAssessment Pydantic** schema. Deterministic fallback if SDK times out.

 **I/O Contract**

Input Context	Output Schema
- NormalizedTransaction - Shared context dictionary (tier1_results, specialist_results)	risk_level (str: 'LOW' 'MEDIUM' 'HIGH' 'CRITICAL') confidence (float [0.0, 1.0]) recommended_action (str: 'APPROVE' 'DECLINE'...) reasoning (Natural language justification) - evidence_summary / risk_factors / mitigating_factors

Agent Profile | Guardrail & Policy Agent

[Execution Tier: GUARDRAIL / POLICY COMPLIANCE]

[Latency Budget: 5ms]

Implementation: `guardrail_agent.py` | `GuardrailAgent`

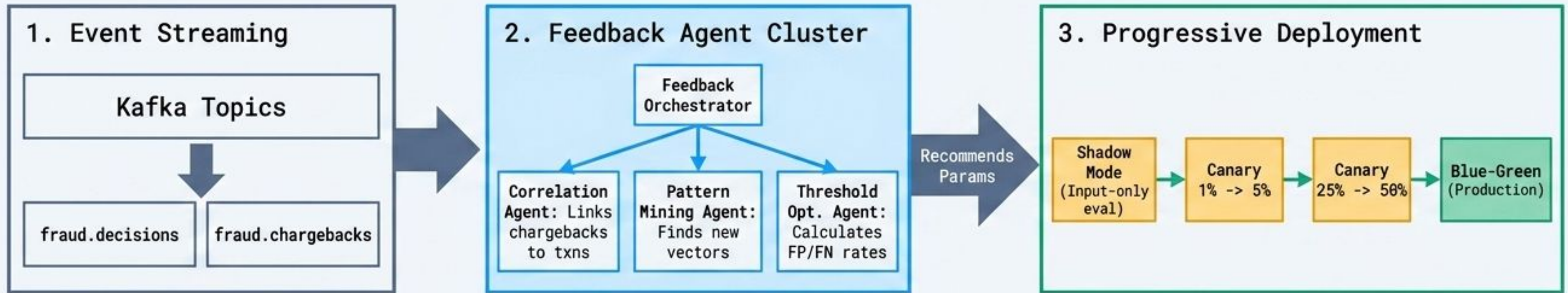
⚙️ Operational Mechanisms	🔧 Mechanisms & Validations	↔️ I/O Contract				
A strict post-execution validator. It guarantees platform stability by intercepting the state before Egress.	1. Evidence Validation: Significant threat flags (blacklisted=True) must possess supporting assertions. 2. Schema Validation: Enforces expected JSON/Pydantic structures; blocks empty/null payloads. 3. Policy/Governance Auditing: Tracks tool hop counts via validate_agent_output(...). 4. Budget Enforcement: Terminates if an agent exceeds its budget by >20% tolerance limit. 5. Hallucination Detection: Bounds validation. Scores must be [0.0, 1.0]; quantities must be non-negative.	<table><tr><th>Input Context</th><th>Output Schema</th></tr><tr><td> • Target agent metadata • Output dictionary • Duration logs • Context counts </td><td> • valid (bool - true if all checks pass) • violations (List of warnings/errors) • evidence_valid (bool) • schema_valid (bool) • policy_valid (bool) • budget_valid (bool) • hallucination_detected (bool) </td></tr></table>	Input Context	Output Schema	• Target agent metadata • Output dictionary • Duration logs • Context counts	• valid (bool - true if all checks pass) • violations (List of warnings/errors) • evidence_valid (bool) • schema_valid (bool) • policy_valid (bool) • budget_valid (bool) • hallucination_detected (bool)
Input Context	Output Schema					
• Target agent metadata • Output dictionary • Duration logs • Context counts	• valid (bool - true if all checks pass) • violations (List of warnings/errors) • evidence_valid (bool) • schema_valid (bool) • policy_valid (bool) • budget_valid (bool) • hallucination_detected (bool)					

Constitutional Governance Rules

All multi-agent reasoning is unconditionally subordinated to the Constitutional Governance Enforcer. The Decision Engine synthesizes outputs into APPROVE, DECLINE, or ESCALATE according to these immutable laws:

Constitutional Policy	Enforcement Mechanism & Verdict Impact
no_policy_override	Governance policies unconditionally override all agent suggestions. Hard blacklists or critical rule breaches (e.g., Sanctioned Countries) result in immediate DECLINE.
no_hallucinated_facts	Any agent outputting out-of-bounds metrics (e.g., Risk Confidence > 1.0 or Distance < 0) is discarded . Triggers a fallback heuristic or defaults to ESCALATE.
evidence_required	Risk flags generated by ML or LLM agents without corresponding database logs in the 'evidence' schema are nullified. Protects against false-positive hallucination.
budget_enforced	Enforces the 100ms SLA. If remaining time drops below the reserve buffer, pending parallel executions are aggressively pruned. Returns conservative heuristic verdict.
escalate_on_uncertainty	Ambiguous inputs, unresolvable discrepancies between Tier-1 and Tier-2 agents, or max_agent_hops: 5 / max_tool_calls: 3 breaches default to ESCALATE (Analyst Queue).

Asynchronous Feedback Cluster & Canary Deployments



The Feedback Loop Breakdown

- **Correlation & Mining:** The async cluster continuously ingests late-arriving outcome data (chargebacks). It mines vectors like 'same_device_many_cards' and attributes errors to specific agent decisions.
- **Threshold Optimization:** Calculates optimal parameter shifts (e.g., safely shifting 'Standard Tier' approval thresholds from 0.10 to 0.12 to increase conversion without raising loss rates).
- **Deployment Pipeline:** Recommendations undergo Governance Agent Review -> Shadow Mode (Evaluated against production traffic without acting) -> Tiered Canary Rollout -> Full Blue-Green promotion.